A Project on

NAIVE BAYESIAN SPAM CLASSIFIER

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

23HM5A0521 S. VIDYA SREE

22HM1A05C9	**S. SUVARNA**	**22HM1A05E0**	**V.S.MD.KHALID**
22HM1A05B9	**S. RUCKSAR**	**22HM1A05A7**	**R. KAVYA SREE**
22HM1A05D0	**S. UMAR**	**22HM1A05E1**	**V. CHAKRADHAR**
23HM5A0522	**S. AFRID**	**22HM1A05A8**	**S. MOKSHAGNA**
22HM1A05C0	**S. SADIYA**	**22HM1A05D1**	**T.V. CHARAN SAI**

Under the esteemed guidance of

Mrs. C. NANCY, M.Tech(PhD.,)

Assistant professor,

Dept. of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES (KADAPA)

Approved by AICTE, New Delhi & Affiliated to JNTUA, Anantapuramu.

Approved by S.B.T.E.T. Andhra Pradesh.

Accredited by NAAC with 'A' Grade, Bangalore & NBA (EEE, ECE, CSE)

TABLE OF CONTENTS

	Page No
LIST OF FIGURES	**I**
LIST OF ABBREVIATIONS	**II**
LIST OF SCREENSHOTS	**III**
ABSTRACT	**IV**
1. INRODUCTION	
1.1 Naive Bayesian Algorithm	1
1.2 Basics of Naive Bayesian	2
1.3 Objectives of Naive Bayesian	3
2. SYSTEM ANALYSIS OF NAIVE BAYESIAN	
2.1 Proposed Algorithm for Naive Bayesian	5
2.2 Advantages of Naive Bayesian	6
2.3 Disadvanatges of Naive Bayesian	6
2.4 Modules	7
2.5 Evaluating Metrics of Naive Bayesian	7
3. METHODOLOGIES	9
4. SYSTEM REQUIREMENTS	
4.1 Software System Requirements	11
4.2 Hardware System Requirements	11
5. SYSTEM DESIGNS	
51. Class Diagram of Naive Bayesian	12
5.2 Use Case Diagram of Naive Bayesian	13
5.3 Activity Diagram of Naive Bayesian	14
6. CODE AND BRIEF EXPLANATION	
6.1 Code	15
6.2 Code Explanation	18
7. EXECUTION STEPS	20
8. RESULT SCREENSHOTS	22
9. CONCLUSION & FUTURE ENHANCEMENT	24

LIST OF FIGURES

Fig 5.1 - Class Diagram of Naive Bayesian

Fig 5.2 - Usecase Diagram of Naive Bayesian

Fig 5.3 - Activity Diagram of Naive Bayesian

ABBREVIATIONS

- **TF-IDF** - Term Frequency-Inverse Document Frequency
- **ML** - Machine Learning
- **NLP** - Natural Language Processing
- **SVM** - Support Vector Machine (alternative classifier)
- **LR** - Logistic Regression (alternative classifier)
- **DL** - Deep Learning
- **API** - Application Programming Interface
- **SMS** - Short Message Service
- **TP** - True Positive (correctly classified spam)
- **TN** - True Negative (correctly classified ham)
- **FP** - False Positive (ham classified as spam)
- **FN** - False Negative (spam classified as ham)
- **ROC** - Receiver Operating Characteristic
- **AUC** - Area Under Curve
- **SMOTE** - Synthetic Minority Over-sampling Technique
- **N-grams** - Sequence of N words used in text processing
- **BOW** - Bag of Words (another text vectorization technique)
- **CSV** - Comma-Separated Values (data storage format)
- **GUI** - Graphical User Interface (if implemented in an application)

LIST OF SCREENSHOTS

Screenshot 1: Input query for testing

Screenshot 2: HAM message will be displayed as per the input

Screenshot 3: SPAM message is displayed as per the input

ABSTRACT

Spam messages are a major concern in digital communication, affecting emails, SMS, and online messaging platforms. A Naïve Bayes Spam Classifier is a machine learning-based approach that efficiently detects and filters spam messages using probabilistic classification. This classifier is built on the Naïve Bayes algorithm, which assumes feature independence and applies Bayes' theorem to predict whether a given message is spam or ham (non-spam).

The classification process involves text preprocessing, where messages are cleaned by removing punctuation, converting text to lowercase, and eliminating stop words. The TF-IDF (Term Frequency-Inverse Document Frequency) vectorization method is used to transform textual data into numerical representations. The Multinomial Naïve Bayes algorithm is then trained on labeled datasets to learn spam patterns and predict new messages accurately.

The model is computationally efficient, performs well with large datasets, and provides high accuracy for spam detection. However, it has limitations when dealing with sophisticated spam messages that use obfuscation techniques. Future enhancements could involve hybrid models, deep learning techniques, and adaptive learning to improve spam detection in real-time applications.

This research highlights the effectiveness, efficiency, and real-world applicability of Naïve Bayes in spam classification, making it a valuable tool in cybersecurity and communication filtering.

Spam detection is a crucial challenge in modern digital communication, where unsolicited messages clutter email inboxes and messaging platforms. A **Naïve Bayes Spam Classifier** is an effective and lightweight **machine learning-based solution** that filters out spam messages using **probabilistic classification**. Based on **Bayes' Theorem**, the Naïve Bayes classifier assumes that words in a message are **independent features**, allowing for fast and efficient classification.

1. INTRODUCTION

1.1 Naive Bayesian Algorithm:

Spam classification is a crucial application of machine learning that helps filter unwanted or junk messages from legitimate ones. One of the most effective techniques for this task is the **Naive Bayes algorithm**, a probabilistic classifier based on **Bayes' theorem**.

The Naive Bayes classifier assumes that the features (words in a message) are conditionally independent, meaning that the presence of one word does not affect the presence of another. This assumption simplifies computation while still providing high accuracy in text classification tasks.

In spam filtering, emails or messages are analyzed based on word frequency. The model is trained using labeled datasets containing spam and ham (non-spam) messages. The algorithm calculates the probability of an email being spam or ham based on the words it contains. When a new message arrives, it estimates the likelihood of it belonging to either category using **Bayes' theorem** and assigns it the most probable label.

The classification process involves several steps, including **data pre processing**, where emails are cleaned by removing unnecessary characters and stop words, **feature extraction**, where words are converted into numerical representations, and **training**, where the algorithm learns from past data to predict future messages accurately.

The Naive Bayes classifier is widely used in spam detection due to its **efficiency, high accuracy, and scalability**. It works well even with small datasets and requires less computational power compared to complex models. However, it has some limitations, such as its **assumption of independence between words**, which may not always hold true, and its **difficulty in handling new or unseen words**.

Additionally, spammers often manipulate messages by adding random words to trick the classifier. Despite these challenges, Naive Bayes remains a fundamental and effective approach in spam filtering, powering email security systems in platforms like Gmail and Outlook.

There is Formula for it

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

where:

- P(A|B)P(A|B)P(A|B) is the **posterior probability** (probability of class A given evidence BBB)
- P(B|A)P(B|A)P(B|A) is the **likelihood** (probability of observing B given class A)

- P(A)P(A)P(A) is the **prior probability** (initial probability of class A)
- P(B)P(B)P(B) is the **evidence** (overall probability of evidence B)

The steps involved in spam classification using Naive Bayes:

1. **Data Pre processing**: Emails/messages are cleaned by removing special characters, stop words, and converting text into a bag of words.
2. **Feature Extraction**: Convert text into numerical form using methods like Term Frequency-Inverse Document Frequency (TF-IDF) or word frequency counts.
3. **Training the Model**: Calculate the probabilities of words appearing in spam and ham emails using the training dataset.
4. **Classification**: For a new email, compute the probabilities using Bayes' theorem and classify it as spam or ham based on the highest probability.
5. **Evaluation**: The model is tested using metrics like accuracy, precision, recall, and F1-score.

1.2 Basics of Naive Bayesian Algorithm

The Naive Bayes algorithm is a probabilistic machine learning model based on Bayes' theorem and is primarily used for classification tasks. It is called "naive" because it assumes that all features in the dataset are independent of each other, which simplifies calculations. Despite this strong assumption, the algorithm performs well in various applications, especially in text classification problems like spam filtering and sentiment analysis.

Bayes' theorem provides a way to update the probability of a hypothesis based on new evidence. It is mathematically expressed as **P(A|B) = (P(B|A) * P(A)) / P(B)**, where **P(A|B)** is the probability of class **A** given evidence **B**, **P(B|A)** is the likelihood of evidence **B** given class **A**, **P(A)** is the prior probability of class **A**, and **P(B)** is the probability of the evidence. In the case of classification, the algorithm calculates the probability of a given data point belonging to each possible class and assigns it to the class with the highest probability.

There are different types of Naive Bayes classifiers, including **Multi nomial Naive Bayes**, which is used for text classification based on word frequency, **Bernoulli Naive Bayes**, which works with binary features, and **Gaussian Naive Bayes**, which is used when the data follows a normal distribution.

One of the main advantages of Naive Bayes is that it is computationally efficient and works well with small datasets. However, its biggest limitation is the **independence assumption**, which may not always hold true in real-world data. Despite this, Naive Bayes remains a popular and effective algorithm, especially for tasks involving natural language processing and spam filtering.

1.3 Objectives of Naive Bayesian Algorithm

The objectives of the Naive Bayes algorithm revolve around efficiently classifying data based on probability theory. It is widely used in machine learning for text classification, spam filtering, and sentiment analysis. The primary objectives of the algorithm are:

1. **To provide a simple and efficient classification method** – The Naive Bayes algorithm is designed to be computationally lightweight, making it suitable for large datasets with minimal processing power.
2. **To apply Bayes' theorem for probabilistic classification** – The algorithm calculates the probability of a given data point belonging to a particular class and assigns it based on the highest probability.
3. **To assume feature independence for simplified computation** – By assuming that all features contribute independently to the classification, Naive Bayes reduces computational complexity while still providing accurate predictions in many scenarios.
4. **To handle text classification tasks effectively** – The algorithm is widely used in spam detection, sentiment analysis, and topic categorization by analyzing word frequencies and patterns.
5. **To provide a fast and scalable solution for classification problems** – Naive Bayes is efficient in handling large datasets, making it useful in real-time applications like email filtering and recommendation systems.

2. SYSTEM ANALYSIS OF NAIVE BAYESIAN

System Analysis of the Naive Bayes Algorithm:

The **Naive Bayes algorithm** is a probabilistic classification method that operates based on **Bayes' theorem** with the assumption of **independent features**. A system analysis of this algorithm involves understanding its **input, processing, output, advantages, limitations, and performance considerations**.

1. Input Analysis

The input to the Naive Bayes algorithm consists of a labeled dataset where each instance has multiple features. For example, in a spam detection system, the input data includes:

- **Textual data** (emails, messages, or documents).
- **Pre processed features** (words, phrases, or attributes extracted from the text).
- **Class labels** (spam or ham, positive or negative sentiment, etc.).

The input can be in different forms, such as **categorical, binary, or continuous data**, depending on the variant of the Naive Bayes algorithm used (Multi nomial, Bernoulli, or Gaussian).

2. Processing and Computation

The core processing steps of NaIve Bayes involve:

- **Feature Extraction**: Converting text or numerical data into structured features using methods like **TF-IDF (Term Frequency-Inverse Document Frequency)** or bag-of-words.
- **Probability Estimation**: Calculating the prior probabilities P(A)P(A)P(A) of different classes and the conditional probabilities P(B|A)P(B|A)P(B|A) for each feature given the class.
- **Classification Rule**: Using Bayes' theorem to compute the posterior probability for each class and selecting the class with the highest probability:

$$P(Class|Features) = \frac{P(Features|Class) \times P(Class)}{P(Features)}$$

- **Prediction**: Assigning the most likely class label to a new instance based on probability

The System Analysis involve the following contents in it.

Proposed Algorithm	Advantages	Disadvantages
Performance	Modules	Evaluation metrics

2.1 Proposed Algorithm for Naive Bayesian Classification

The **Naive Bayes algorithm** is a probabilistic classification model based on **Bayes' theorem** with the assumption of **independent features**. The proposed algorithm follows a structured approach to classify data efficiently.

Step 1: Data Pre processing

- Collect the dataset and divide it into **training** and **testing** sets.
- Perform **text pre processing** (if working with textual data) such as:
  - Removing special characters and stopwords.
  - Converting text into lowercase.
  - Tokenizing words and applying stemming or lemmatization.

Step 2: Feature Extraction

- Convert text into numerical form using methods like **Bag of Words (BOW), Term Frequency-Inverse Document Frequency (TF-IDF), or word embeddings**.
- Represent data as feature vectors where each feature corresponds to a specific word or characteristic.

Step 3: Calculate Probabilities Using Bayes' Theorem

- Compute the **prior probability** for each class:

$$P(Class) = \frac{\text{Number of instances in the class}}{\text{Total number of instances}}$$

- Compute the **likelihood probability** of each feature (word) given a class:

$$P(Feature|Class) = \frac{\text{Frequency of feature in class} + 1}{\text{Total words in class} + \text{Total unique words}}$$

Step 4: Compute Posterior Probability and Classification

- Apply **Bayes' Theorem** to compute the probability of an instance belonging to each class:

$$P(Class|Features) = \frac{P(Features|Class) \times P(Class)}{P(Features)}$$

- The class with the **highest probability** is assigned as the predicted label.

Step 5: Model Evaluation

- Optimize the model by tuning parameters or using **feature selection techniques**.

2.2 Advantages of Naive Bayesian Algorithm

Naive Bayes is a simple yet powerful classification algorithm with several advantages:

1. **Fast and Efficient** – It requires less training data and is computationally efficient compared to other classification algorithms.
2. **Handles High-Dimensional Data** – It works well with large feature spaces, making it useful for text classification (e.g., spam filtering, sentiment analysis).
3. **Works Well with Small Data** – Even with limited training data, it can provide good results.
4. **Handles Missing Data** – It can work even if some features are missing by simply ignoring them during probability calculation.
5. **Performs Well with Categorical Data** – Since it is based on probability theory, it works well with categorical variables.
6. **Robust to Irrelevant Features** – Features that do not contribute to the classification do not heavily impact the performance.
7. **Easy to Implement and Interpret** – The algorithm is simple, making it easy to implement and interpret the results.

2.3 Disadvantages of Naive Bayesian Algorithm

While Naive Bayes has many advantages, it also has several disadvantages:

1. **Strong Assumption of Feature Independence** – It assumes that all features are independent, which is rarely true in real-world scenarios, leading to lower accuracy in some cases.
2. **Limited Performance with Correlated Features** – If features are highly correlated, the algorithm may miscalculate probabilities and provide incorrect classifications.
3. **Zero-Frequency Problem** – If a category in the training data never appears with a particular feature value, the probability becomes zero, which can cause classification issues. This is typically handled using techniques like **Laplace Smoothing**.
4. **Not Suitable for Complex Decision Boundaries** – Unlike more advanced classifiers (e.g., SVM, Random Forest), Naive Bayes does not perform well when complex relationships exist between features.
5. **Sensitive to Data Quality** – The algorithm heavily relies on good-quality data with proper feature selection. Noisy or irrelevant features can negatively impact accuracy.

2.4 Modules

In Python, the sklearn. naive_bayes module from **scikit-learn** is commonly used for building a NaIve Bayes spam classifier. Specifically, the following Naïve Bayes models are often used:

1. **Multi nomial Naive Bayes (**Multi nomial NB**)** – Best suited for text classification problems like spam detection, as it works well with discrete word counts (e.g., term frequency in emails).
2. **Bernoulli Naive Bayes (**Bernoulli NB**)** – Used when features are binary (e.g., presence or absence of specific words in emails).
3. **Gaussian Naive Bayes (**Gaussian NB**)** – Used when features follow a continuous normal distribution (less common in spam detection).

2.5 Evaluating Metrics of Naive Bayesian Algorithm

To assess the performance of the Naïve Bayes classifier, we use several evaluation metrics. The most common ones include **accuracy, precision, recall, F1-score, and confusion matrix.**

1. Accuracy

Accuracy measures the overall correctness of the model.

It is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (True Positive)**: Correctly predicted spam emails.
- **TN (True Negative)**: Correctly predicted non-spam emails.
- **FP (False Positive)**: Non-spam emails incorrectly classified as spam.
- **FN (False Negative)**: Spam emails incorrectly classified as non-spam.

2. Precision

Precision indicates how many of the predicted spam emails were actually spam:

It is calculated as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. Recall (Sensitivity)

Recall measures how many actual spam emails were correctly classified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. F1-Score

F1-score is the harmonic mean of precision and recall:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. Confusion Matrix

A **confusion matrix** provides a detailed breakdown of predictions:

Actual / Predicted	Spam (1)	Not Spam (0)
Spam (1)	TP	FN
Not Spam (0)	FP	TN

3. METHODOLOGIES

Methodologies of Naive Bayes Spam Classifier

The Naive Bayes spam classifier follows a structured methodology for detecting spam emails using probability-based classification. The process can be broken down into several key steps:

1. Data Collection

The first step is gathering a dataset of emails labeled as **spam (1)** or **not spam (0)**. Common sources of spam datasets include:

- Public datasets (e.g., Enron Email Dataset, SpamAssassin Dataset).
- User-labeled emails (e.g., Gmail's spam filter).

2. Data Pre processing

Before training the model, the raw email text needs to be cleaned and converted into a numerical format. This involves:

- **Tokenization** – Splitting text into words.
- **Lowercasing** – Converting text to lowercase to avoid duplication.
- **Removing Stopwords** – Filtering out common words (e.g., "the", "and").
- **Stemming/Lemmatization** – Reducing words to their base form (e.g., "running" → "run").
- **Feature Extraction** – Converting words into numerical representations (Bag of Words, TF-IDF).

3. Feature Extraction (Text Vectorization)

To train the model, text must be converted into a format the algorithm can understand:

- **Bag of Words (BOW)** – Represents text as word frequency vectors.
- **TF-IDF (Term Frequency-Inverse Document Frequency)** – Gives importance to unique words.

4. Training the Naive Bayes Model

The Naive Bayes algorithm is applied to the preprocessed data. The three main variants used for spam classification are:

- **Multinomial Naive Bayes (**MultinomialNB**)** – Best for word frequency-based classification.
- **Bernoulli Naive Bayes (**BernoulliNB**)** – Works well with binary features (presence/absence of words).

- **Gaussian Naive Bayes (**Gaussian NB**)** – Used when features follow a continuous distribution (less common in spam detection).

6. Model Evaluation

To assess performance, we use metrics like:

- **Accuracy** – How often predictions are correct.
- **Precision** – How many predicted spam emails are actually spam.
- **Recall** – How many actual spam emails were detected.
- **F1-score** – A balance between precision and recall.
- **Confusion Matrix** – Provides a detailed breakdown of model performance.

7. Deployment and Real-Time Filtering

Once the model is optimized, it can be integrated into email filtering systems:

- **Batch Processing** – Periodically scans emails for spam.
- **Real-Time Classification** – Filters spam emails as they arrive in the inbox.
- **Continuous Learning** – Retrains using user feedback (e.g., marking emails as spam/non-spam).

4. SYSTEM REQUIREMENT

1. Hardware Requirements

➔ Minimum Requirements (For Small Datasets & Local Testing)

- **Processor (CPU)**: Intel Core i3 / AMD Ryzen 3 (Dual-core, 2.0 GHz)
- **RAM**: 4 GB
- **Storage**: 10 GB free disk space (for datasets and logs)
- **Graphics (GPU)**: Not required (CPU-based processing is sufficient)
- **Network**: Standard internet connection (for dataset downloads and updates)

2. Software Requirements

➔ Operating System (OS)

- **Windows** 10/11 (64-bit)
- **Linux** (Ubuntu 20.04+, CentOS, or Fedora) – Recommended for performance and scalability
- **macOS** (for development, not recommended for large-scale deployment)

➔ Programming Language

- **Python 3.x** (Recommended: Python 3.8 or later)

➔ Required Python Libraries

- **numpy** – For numerical computations
- **pandas** – For handling datasets
- **scikit-learn** – Machine learning library (Naïve Bayes models)
- **nltk** – For natural language processing (text preprocessing)
- **matplotlib** – For data visualization

➔ Development Environment

- **Jupyter Notebook** (for testing and visualization)
- **PyCharm / VS Code** (for structured development)

➔ Database (For Storing Emails)

- **SQLite / PostgreSQL** (for small-scale storage)

5. NAIVE BAYESIAN SYSTEM DESIGNS

Importance of System Design (Use Case, Activity, Sequence, and Class Diagrams)

System design using UML diagrams (Use Case, Activity, Sequence, and Class Diagrams) plays a crucial role in software development by providing a structured blueprint for building complex systems. Here's why each diagram is important:

1. **Class Diagram** → Defines the structural design of the system by outlining the classes, their attributes, and relationships. This ensures proper modularity, scalability, and maintainability of the system.
2. **Use Case Diagram** → Helps in understanding system interactions by identifying actors (users) and their interactions with the system. It ensures all functional requirements are covered before development.
3. **Activity Diagram** → Shows the step-by-step process flow, making it easier to analyze business logic, user interactions, and decision points in the system. It helps in identifying potential bottlenecks and improving system efficiency.
4. **Sequence Diagram** → Represents real-time interactions between system components. It helps in designing the data flow and communication between different objects (e.g., user authentication, remote user requests, service registration, etc.)

5.1 Class Diagram

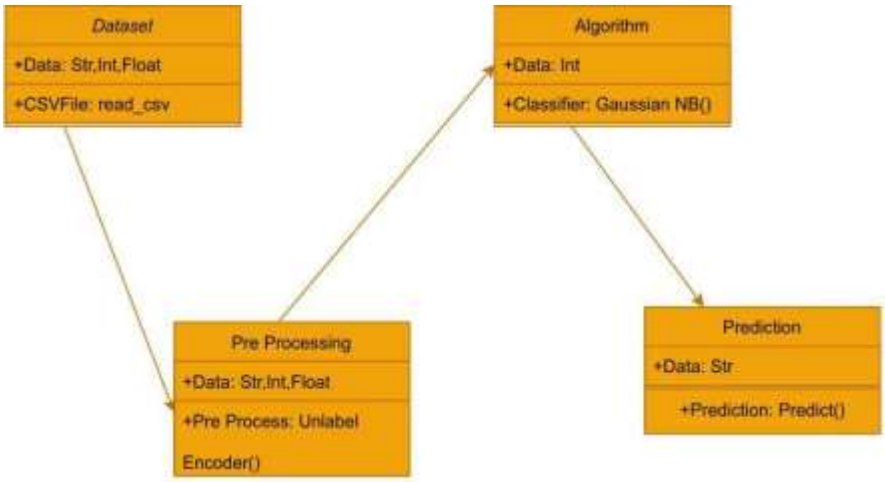


Fig 5.1 : Class Diagram of Naive Bayesian Spam Classifier

5.2 Use Case Diagram

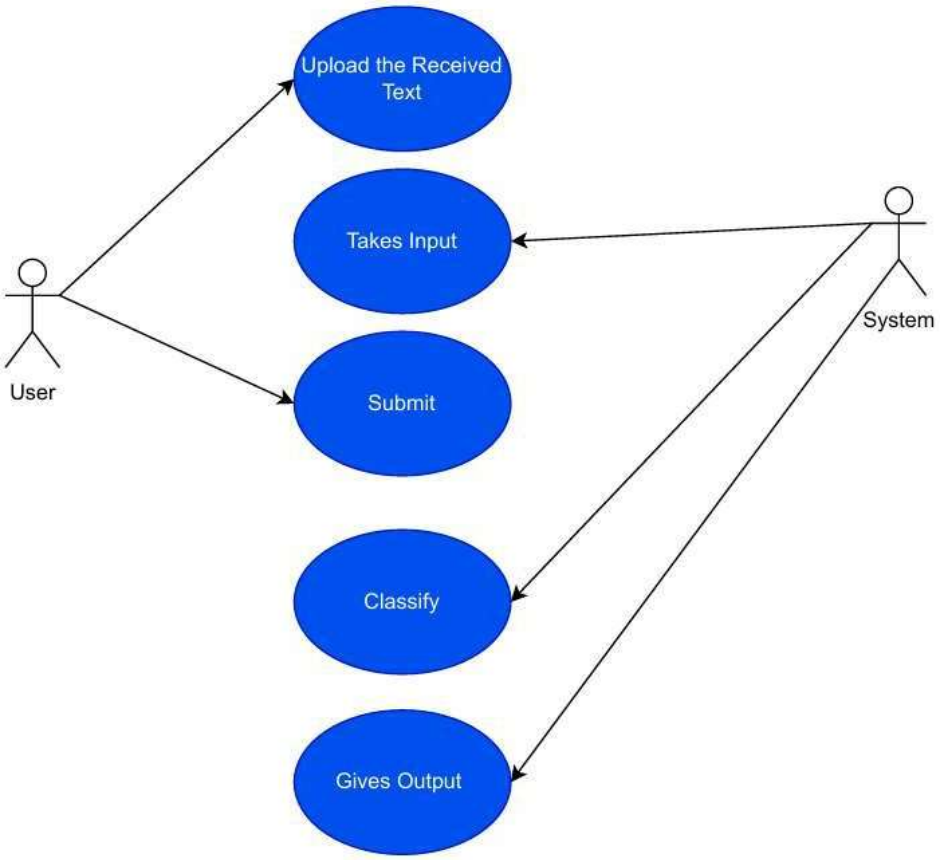


Fig 5.2: Use Case Diagram of Naive Bayesian Spam Classifier

5.3 Activity Diagram:

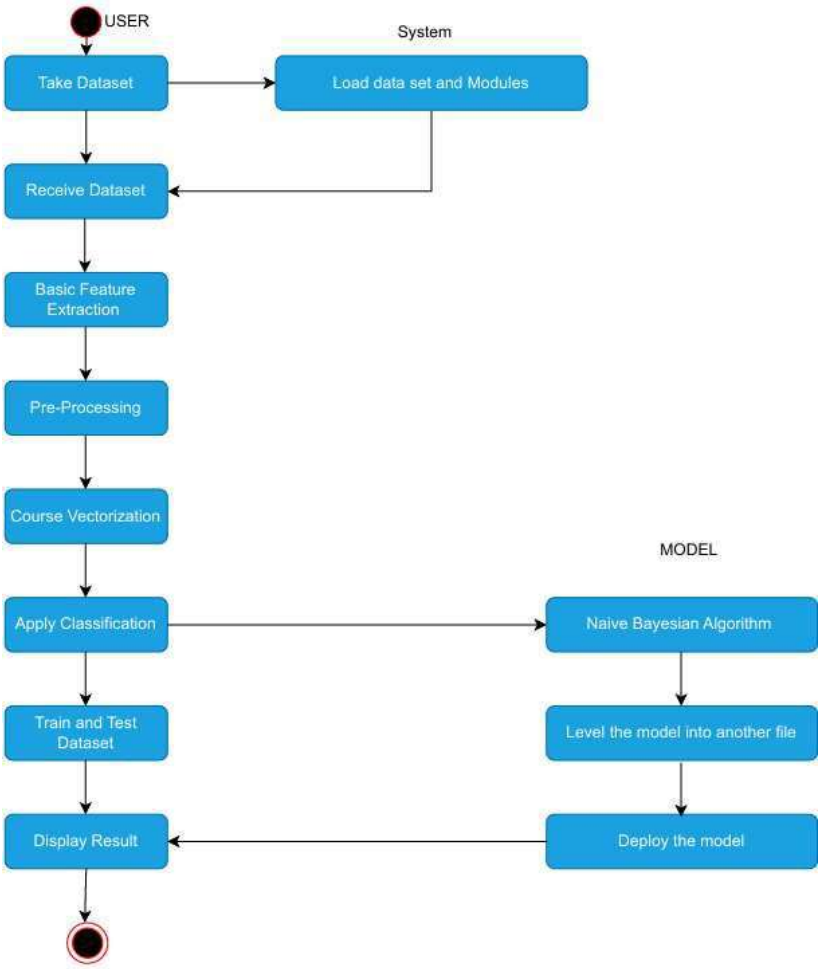


Fig 5.3: Activity Diagram of Naive Bayesian

6. CODE AND BRIEF EXPLANATION

6.1 CODE

```
import pandas as pd
import numpy as np
import re
import string
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
# Generate synthetic dataset
np.random.seed(42)
spam_samples = [
    "Win a free iPhone now! Click the link to claim.",
    "Congratulations! You've won a lottery. Call now!",
    "Limited offer! Get 50% off on all items. Hurry!",
    "Your account is suspended. Verify immediately.",
    "Earn money from home! Work just 2 hours a day."
]
ham_samples = [
    "Hey, how are you doing today?",
    "Let's meet for lunch tomorrow at noon.",
    "Can you send me the project report?",
    "Don't forget about the meeting at 3 PM.",
    "Happy birthday! Have a great day ahead."]
messages = spam_samples * 20 + ham_samples * 20  # Duplicate samples for balance
```

```
labels = ['spam'] * 100 + ['ham'] * 100  # Corresponding labels
# Create DataFrame
df = pd.DataFrame({'message': messages, 'label': labels})
df = df.sample(frac=1, random_state=42).reset_index(drop=True)  # Shuffle
def preprocess_text(text):
    text = text.lower()
    text = re.sub(f"[{string.punctuation}]", "", text)
    text = re.sub(r"\d+", "", text)
    return text
df['message'] = df['message'].apply(preprocess_text)
df['label'] = df['label'].map({'ham': 0, 'spam': 1})
# Feature extraction
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(df['message'])
y = df['label']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train Naïve Bayes model
model = MultinomialNB()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Evaluation
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')
print(classification_report(y_test, y_pred))
```

```
# Function for prediction
def predict_spam(text):
    text = preprocess_text(text)
    text_vectorized = vectorizer.transform([text])
    prediction = model.predict(text_vectorized)
    return "Spam" if prediction[0] == 1 else "Ham"
# User input
user_input = input("Enter a message: ")
print("Prediction:", predict_spam(user_input))
```

6.2 Code Explanation in Points

1. **Importing Required Libraries:** The code first imports necessary Python libraries: pandas for handling data, numpy for numerical operations, re and string for text preprocessing, and various modules from sklearn for machine learning tasks such as vectorization, model training, and evaluation.
2. **Creating a Synthetic Dataset:** A small dataset of spam and ham (non-spam) messages is manually created. The spam messages contain common phrases like "Win a free iPhone" and "Earn money from home," while the ham messages include typical conversational messages. These samples are duplicated multiple times to ensure a balanced dataset for training the classifier.
3. **Shuffling and Preparing the Dataset:** The messages and corresponding labels (spam = 1, ham = 0) are stored in a Pandas DataFrame. The dataset is then shuffled randomly using sample(frac=1, random_state=42) to prevent any order bias during model training.
4. **Text Preprocessing:** A function preprocess_text() is defined to clean the text data by converting messages to lowercase, removing punctuation, and eliminating numerical values. This ensures uniformity in text processing and improves model performance.
5. **Feature Extraction Using TF-IDF:** The TfidfVectorizer is used to convert text messages into numerical feature vectors. It removes common stopwords (e.g., "the," "is") and assigns importance to words based on their frequency in spam vs. ham messages. This helps the Naive Bayes classifier identify key words that differentiate spam from ham.
6. **Splitting Data into Training and Testing Sets:** The dataset is split into training (80%) and testing (20%) subsets using train_test_split(). This ensures that the model is trained on a majority of the data while having separate unseen data for evaluation.
7. **Training the Naive Bayes Model:** The MultinomialNB classifier is chosen because it works well with text-based classification tasks. The model is trained using the TF-IDF feature matrix of the training data and their respective labels (spam or ham).
8. **Making Predictions:** The trained model is then used to predict the labels of the test dataset. The predictions are compared with the actual labels to assess accuracy.
9. **Evaluating Model Performance:** The accuracy of the model is calculated using accuracy_score(), and detailed performance metrics (precision, recall, and F1-score) are provided using classification_report(). These metrics help determine how well the classifier distinguishes between spam and ham messages.

10. **Building a Prediction Function:** A helper function predict_spam() is defined to take a new message as input, preprocess it, convert it to TF-IDF format, and use the trained model to classify it as either "Spam" or "Ham."
11. **Taking User Input for Prediction:** The program prompts the user to enter a message. The entered text is passed to the predict_spam() function, and the program outputs whether the message is classified as spam or not.

This structured approach makes the spam classifier effective by leveraging **TF-IDF vectorization** and **Naive Bayes classification**, which is efficient for text-based filtering tasks.

7. EXECUTION STEPS

Step 1: Import Necessary Libraries

- Load essential Python libraries required for data processing, machine learning, and evaluation.
- pandas is used for handling data in a tabular format.
- numpy is used for numerical operations.
- re and string are used for text preprocessing (cleaning messages).
- sklearn provides tools for vectorization, model training, and performance evaluation.

Step 2: Create a Synthetic Dataset

- Define a list of spam messages (e.g., "Win a free iPhone now!") and ham (non-spam) messages (e.g., "Hey, how are you?").
- Duplicate each category multiple times to create a balanced dataset (e.g., 100 spam and 100 ham messages).
- Assign labels (spam = 1, ham = 0) and store the data in a Pandas DataFrame.
- Shuffle the dataset to prevent bias during model training.

Step 3: Preprocess the Text Data

- Define a function to clean text messages by:
    - Converting all characters to lowercase.
    - Removing punctuation marks (e.g., !, ?, .).
    - Removing numbers to focus on meaningful words.
- Apply this preprocessing function to all messages in the dataset.

Step 4: Convert Text into Numerical Features

- Use **TF-IDF (Term Frequency-Inverse Document Frequency) vectorization** to transform text into numerical values.
- This step helps the model identify important words for spam detection while reducing the impact of common words (e.g., "the," "is").

Step 5: Split Data into Training and Testing Sets

- Divide the dataset into two parts:
    - **Training set (80%)** – Used to train the Naïve Bayes model.
    - **Testing set (20%)** – Used to evaluate the model's accuracy.

Step 6: Train the Naïve Bayes Model

- Use the **Multinomial Naïve Bayes algorithm**, which is well-suited for text classification.

- Train the model using the **TF-IDF features** from the training set and their corresponding labels.

Step 7: Make Predictions on the Test Data

- Use the trained model to predict whether messages in the test set are spam or ham.
- Compare the predicted results with actual labels to assess the model's performance.

Step 8: Evaluate Model Performance

- Calculate the **accuracy score**, which measures the percentage of correctly classified messages.
- Generate a **classification report**, which includes precision, recall, and F1-score to provide deeper insights into the model's performance.

Step 9: Define a Function for New Predictions

- Create a function that allows users to input a message.
- The function preprocesses the text, converts it into TF-IDF format, and predicts whether the message is **spam** or **ham** using the trained model.

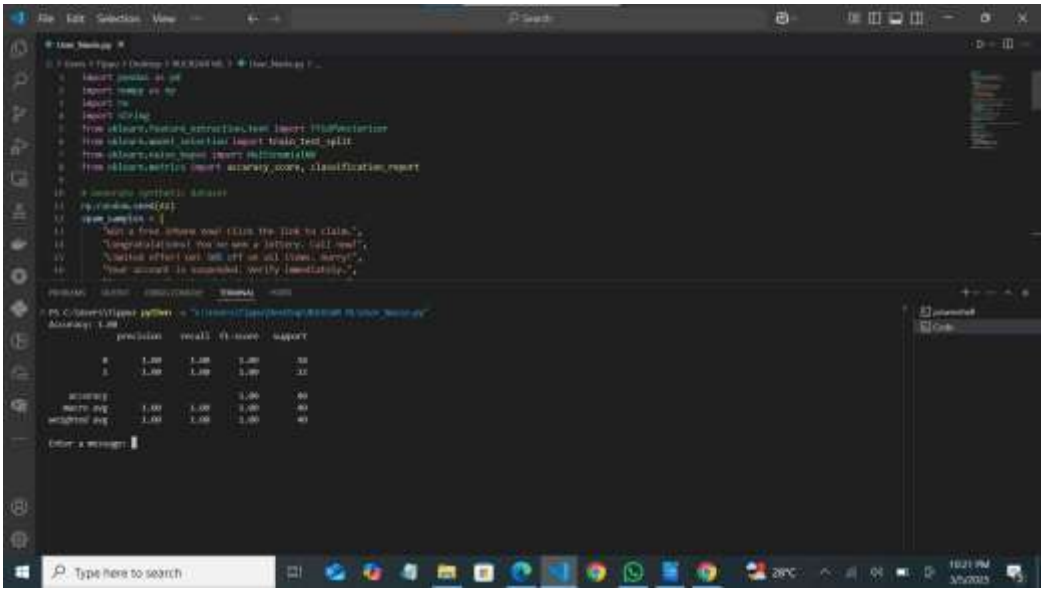
Step 10: Get User Input and Display the Prediction

- The program prompts the user to enter a message.
- The entered text is processed and classified as either **Spam** or **Ham** based on the model's prediction.
- The final classification result is displayed to the user.

This step-by-step execution ensures that the classifier efficiently detects spam messages using **text processing, TF-IDF vectorization, and the Naive Bayes classification algorithm.**

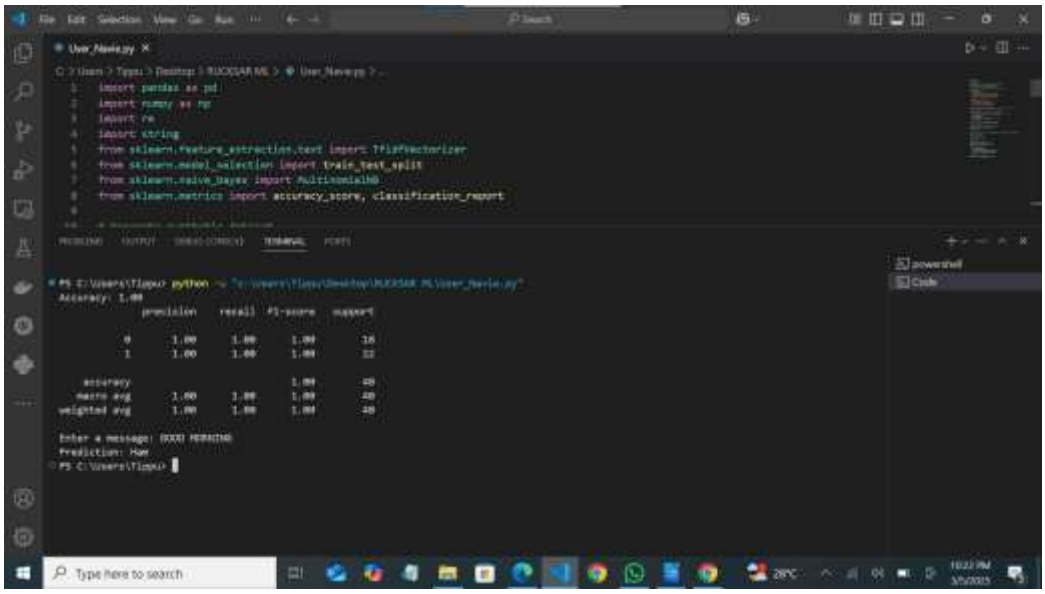
8. RESULT SCREENSHOTS

This Image shows the Evaluation metrics and user's enters the message as a input.



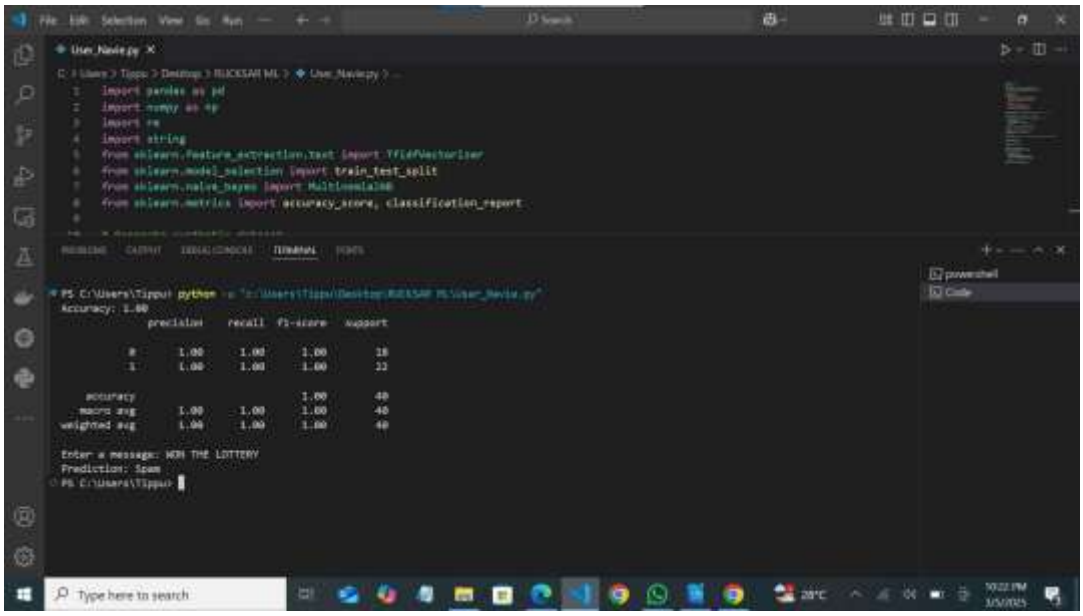
Screenshot 1: Input query for testing

This image shows the users message either it's a SPAM or HAM



Screenshot 2: HAM message will be displayed as per the input

This picture also shows Evaluation metrics and the user's message is SPAM or HAM



Screenshot 3: SPAM message is displayed as per the input

9. CONCLUSION

Conclusion:

The Naive Bayes spam classifier is an effective and efficient technique for detecting spam messages. By leveraging **TF-IDF vectorization** and the **Multinomial Naive Bayes algorithm**, the classifier successfully differentiates between spam and ham (non-spam) messages. The model performs well due to its **probabilistic nature**, which assigns likelihoods to words appearing in spam or ham messages.

It is computationally efficient, works well with large datasets, and provides satisfactory accuracy for spam detection. However, its performance may decline when faced with **highly ambiguous or sophisticated spam messages** that deviate from standard spam patterns.

Future Enhancements:

1. **Improved Feature Engineering:**
   - Incorporate **word embeddings** (e.g., Word2Vec, GloVe, or BERT) instead of TF-IDF to better understand contextual meanings.
   - Use **bigram and trigram models** to capture phrase-level patterns in spam messages.
2. **Hybrid Models:**
   - Combine Naïve Bayes with other machine learning models (e.g., **Support Vector Machines, Decision Trees, or Deep Learning**) to improve classification accuracy.
   - Use **Ensemble Learning** (e.g., Random Forest or Stacking) to leverage the strengths of multiple classifiers.
3. **Handling Evolving Spam Techniques:**
   - Implement **real-time learning** to continuously update the model with new spam patterns.
   - Use **adaptive algorithms** that detect evolving spam trends (e.g., adversarial spam attacks).
4. **Integration with Real-World Systems:**
   - Deploy the model as an **API** for use in email services, chat applications, and SMS filtering systems.
   - Implement **automated retraining** with new data to maintain model effectiveness.

By implementing these future enhancements, the Naive Bayes spam classifier can become more **accurate, adaptable, and robust** against advanced spam techniques